Mohamed bin Zayed University of Artificial Intelligence

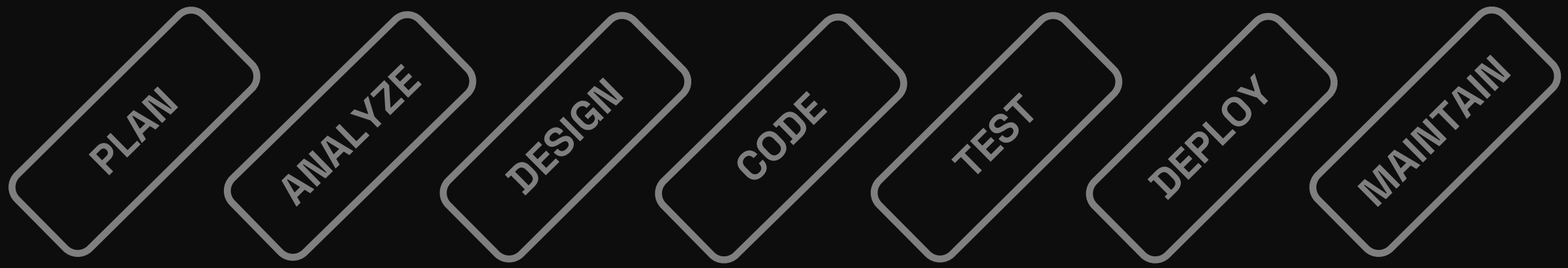
Lecture 6

Architecture design 1

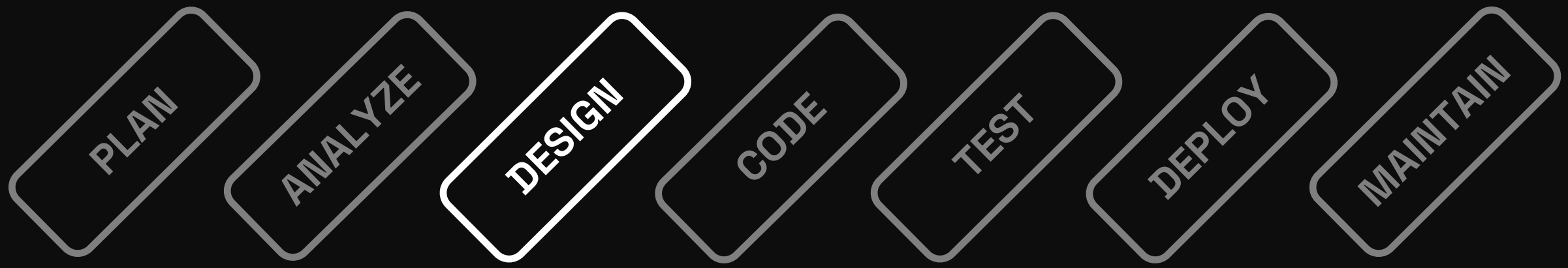
Fabio Pizzati

AI1220: Software, web, mobile engineering

Software development life cycle



Software development life cycle



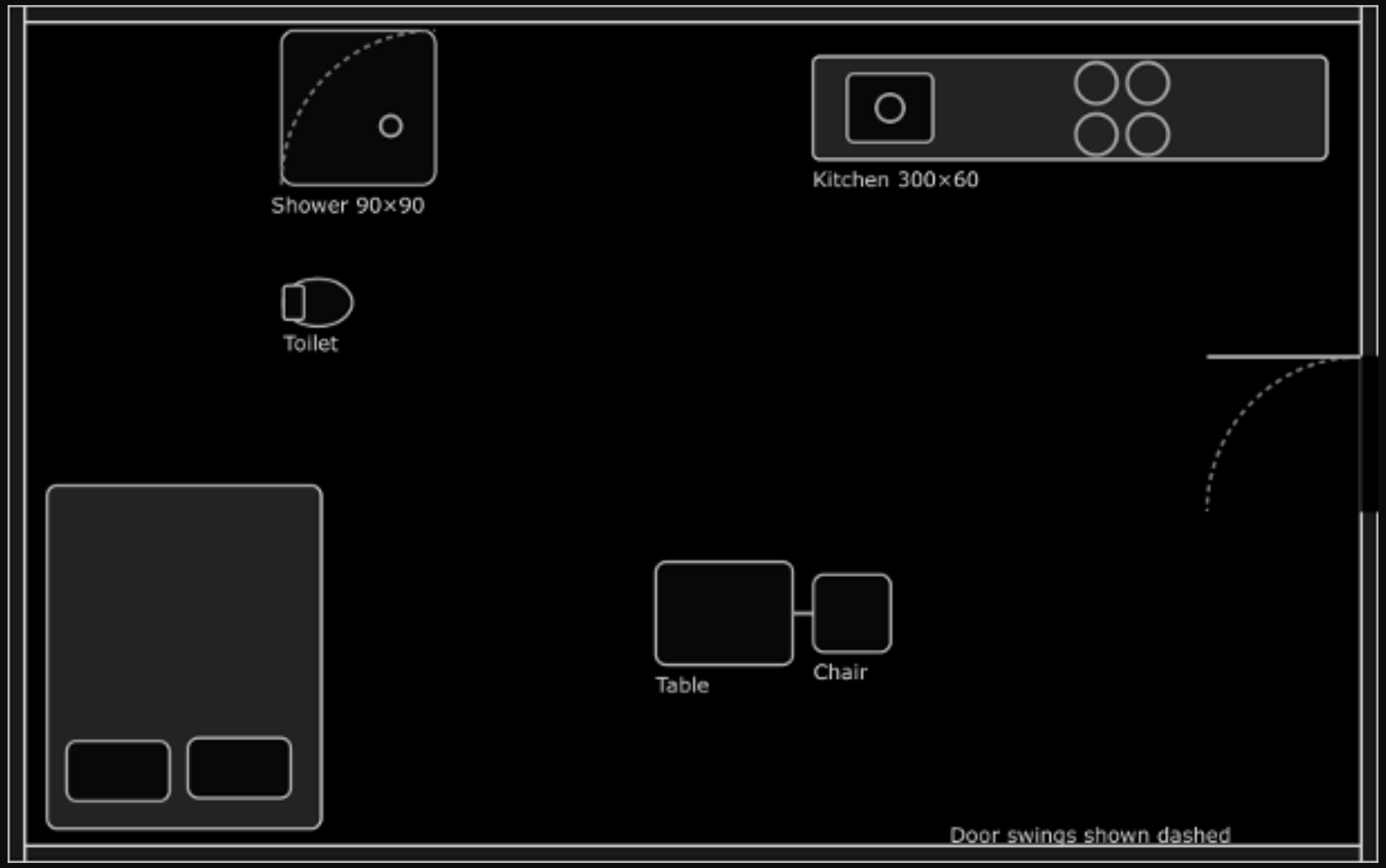
We're building a house

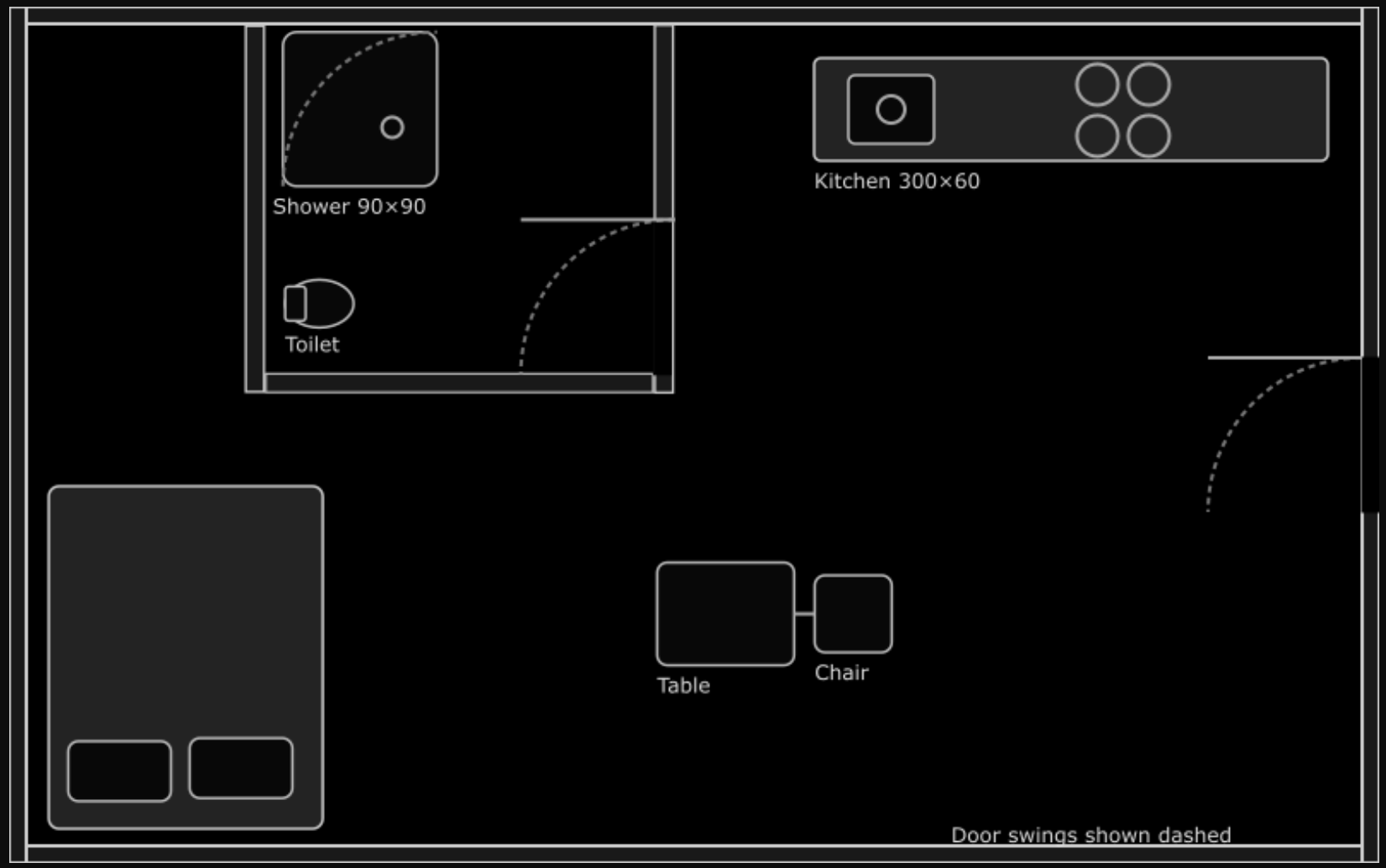
Do we <u>need</u> design?

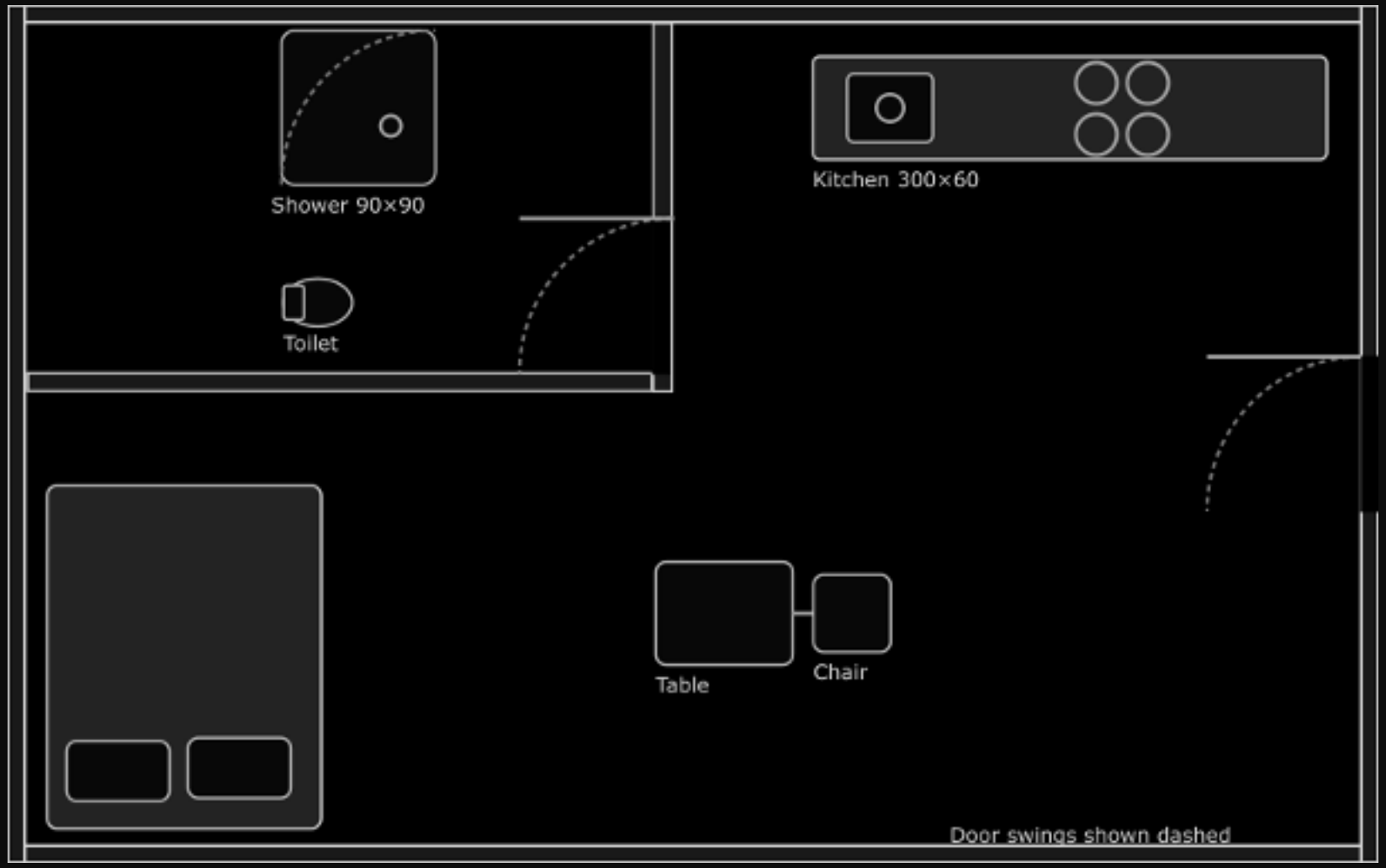
House building guidelines

We just need a bathroom, a kitchen, and a living space.
The bathroom is separated; the rest is in a single room.
Built it fast! We need to deliver.

Door swings shown dashed


Shower 90×90
Kitchen 300×60
Toilet
Table
Chair
Door swings shown dashed

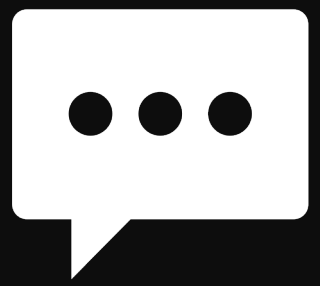

Shower 90×90
Toilet
Kitchen 300×60
Table
Chair
Door swings shown dashed


Shower 90×90
Toilet
Kitchen 300×60
Table
Chair
Door swings shown dashed

r/zillowgonewild

https://www.zillow.com/homedetails/1019-1-2-E-Commercial-Ave-Anaconda-MT-59711/78129951_zpid/

Not convinced?



Stakeholder Communication

System Analysis

Reuse

What is design?

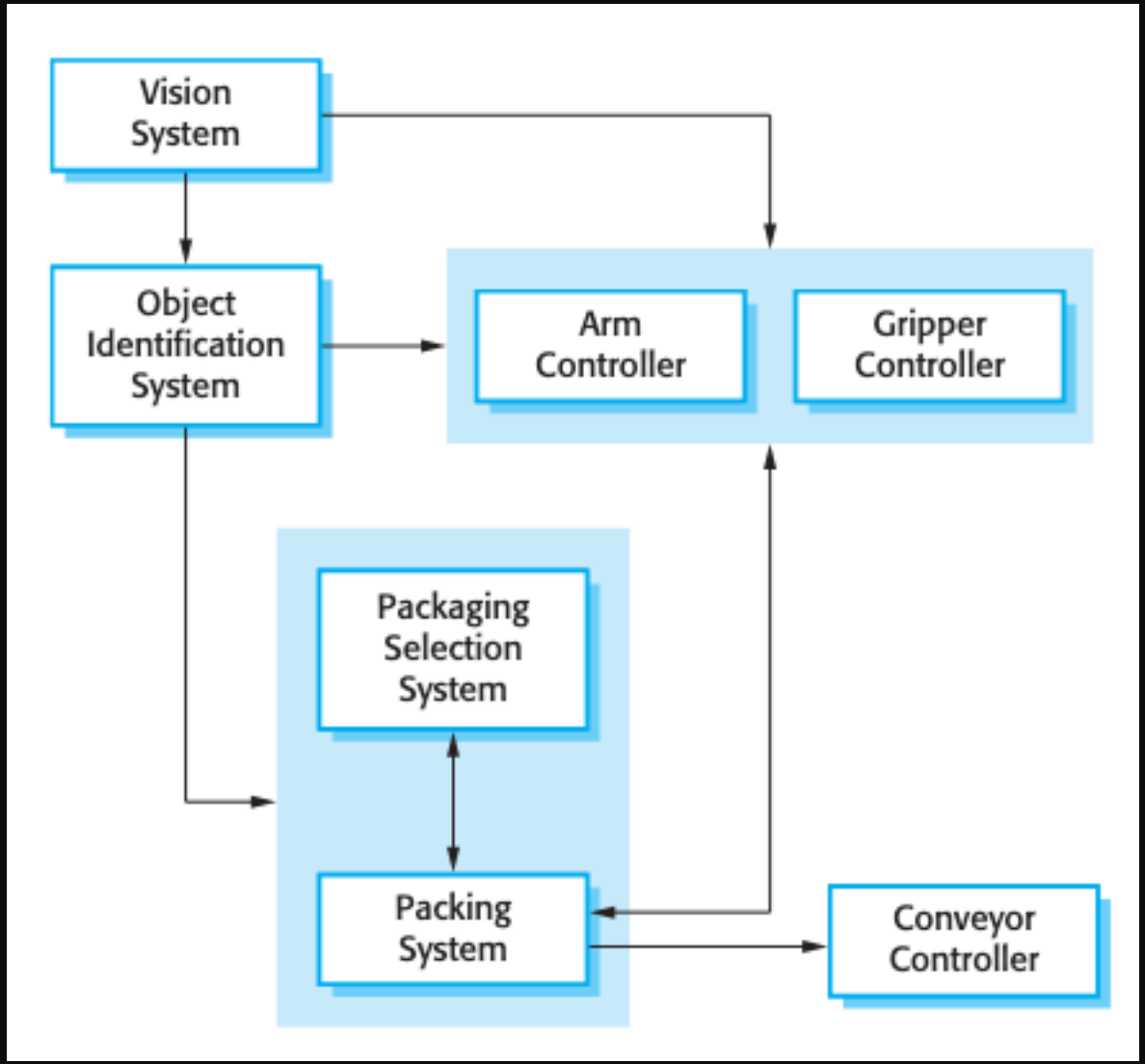
What is design?

Components & interactions!

Architecture Design

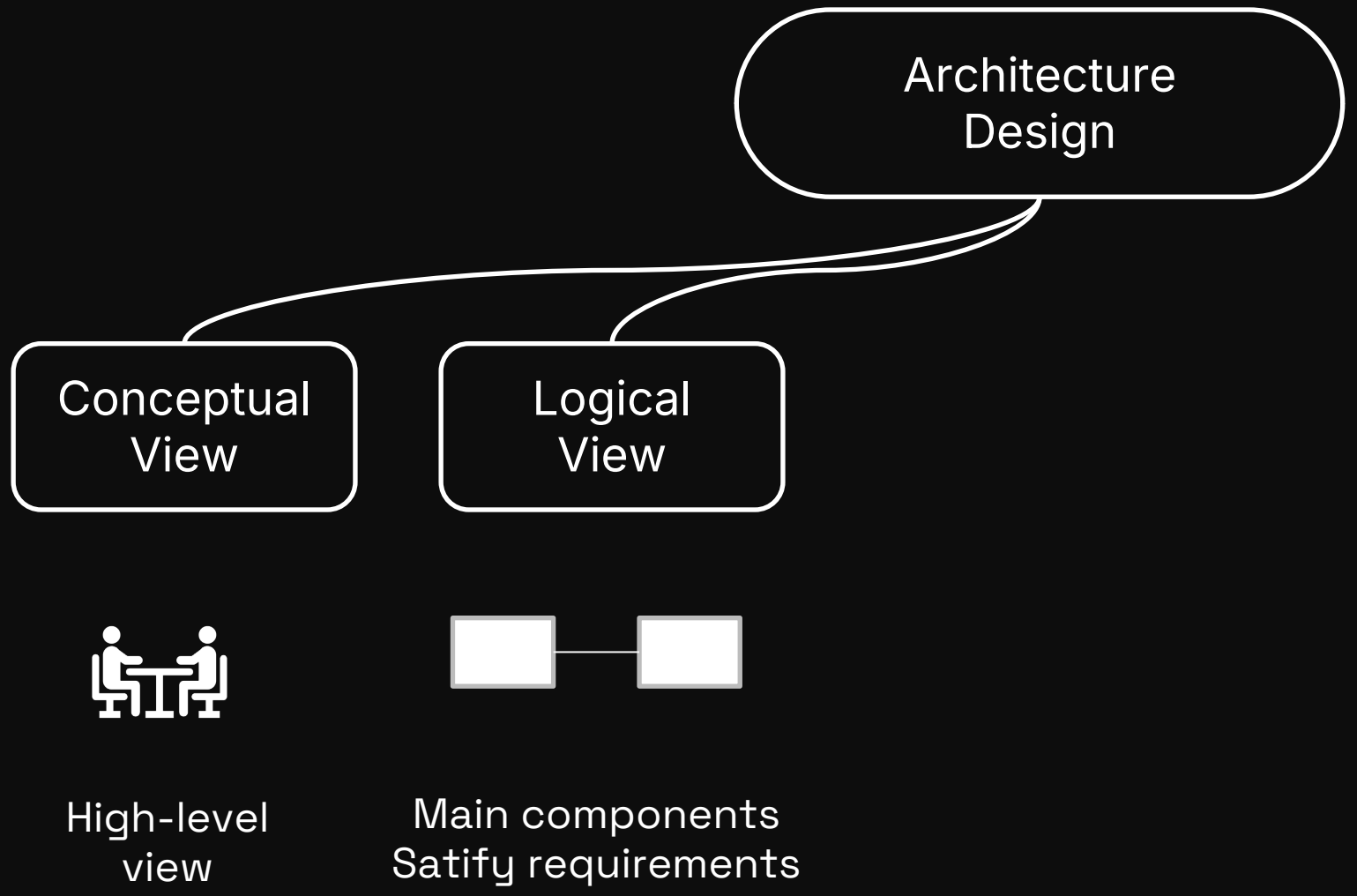
Conceptual View

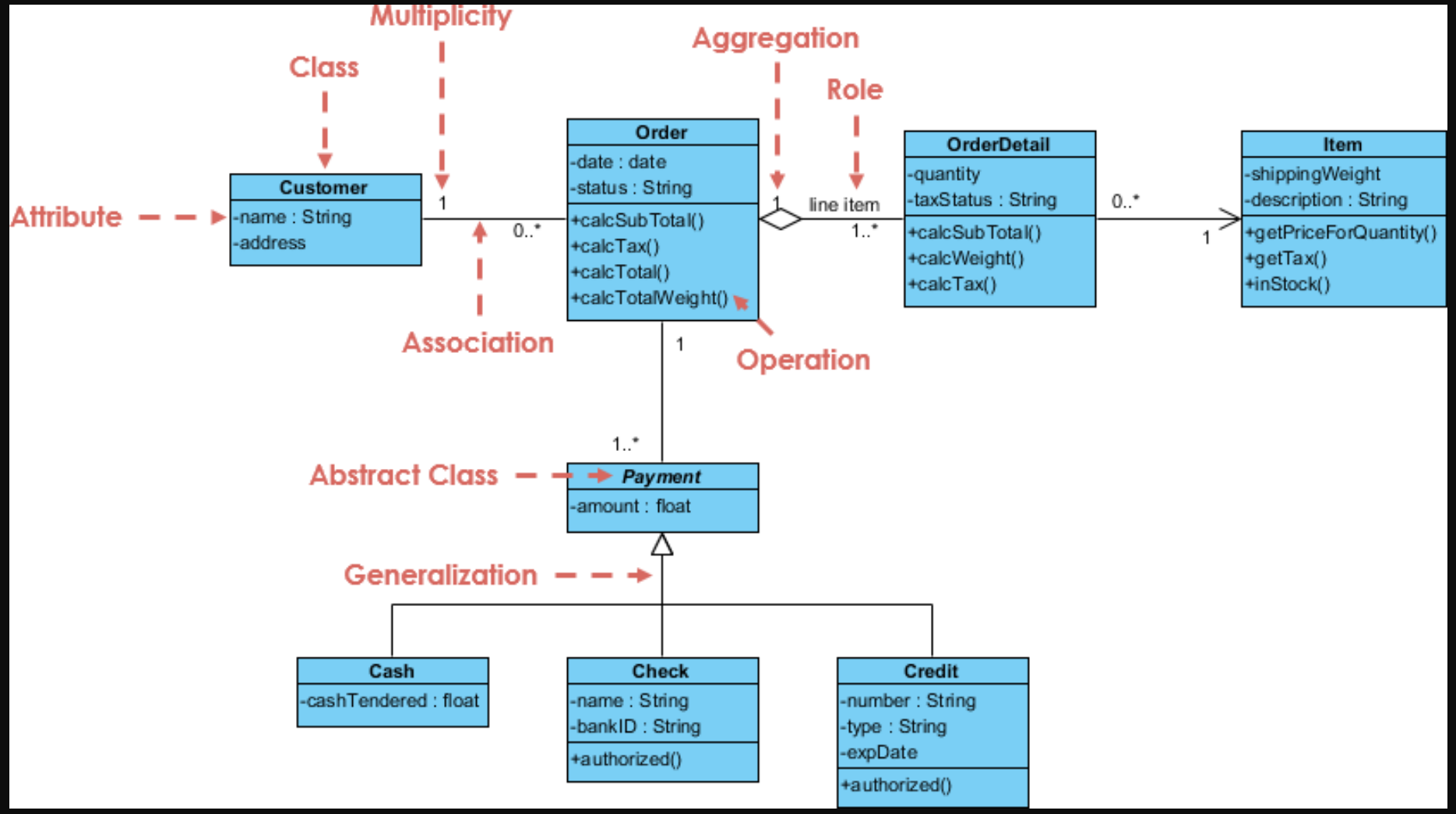
High-level view



Block diagrams

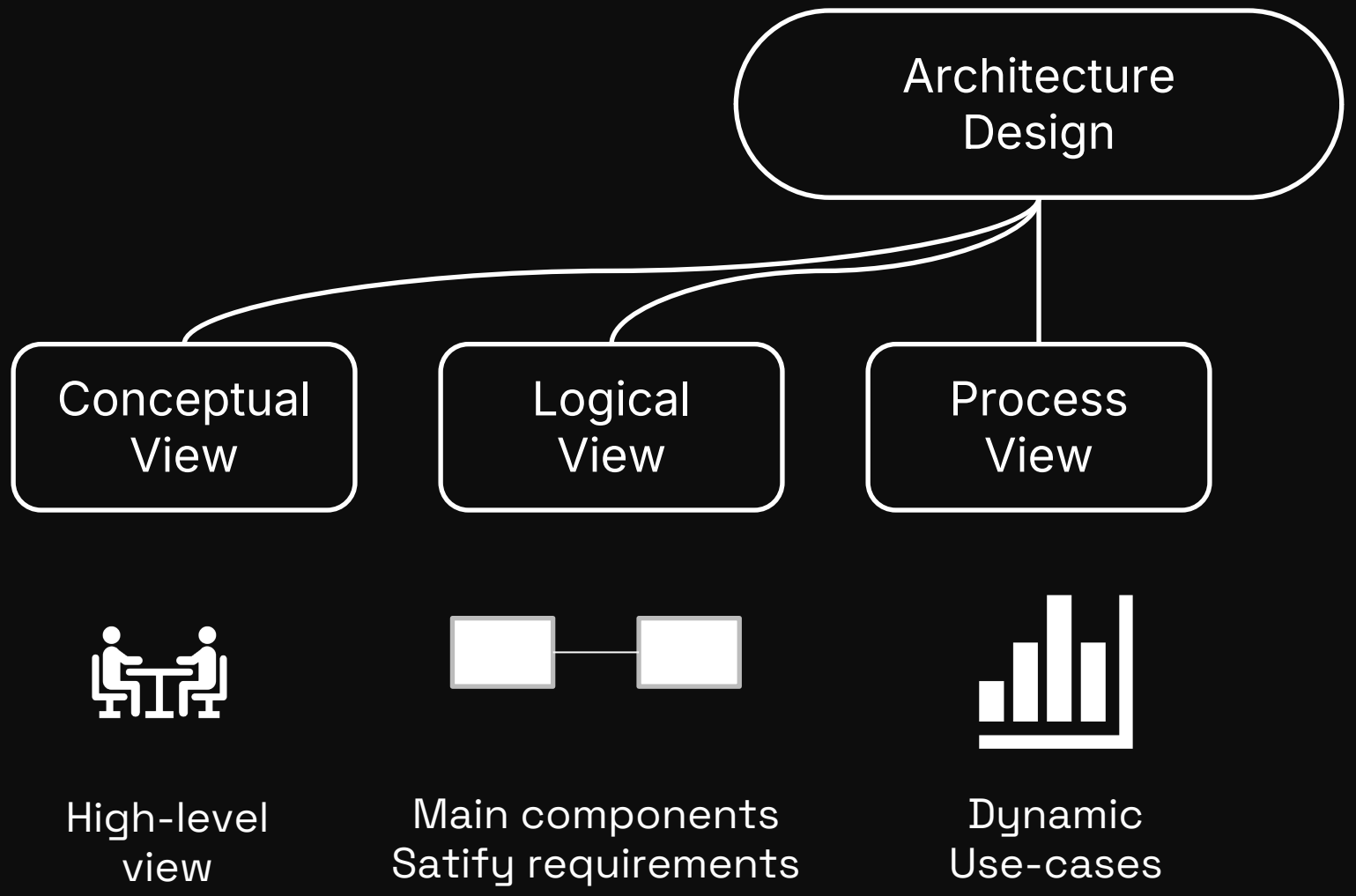
High-level overview

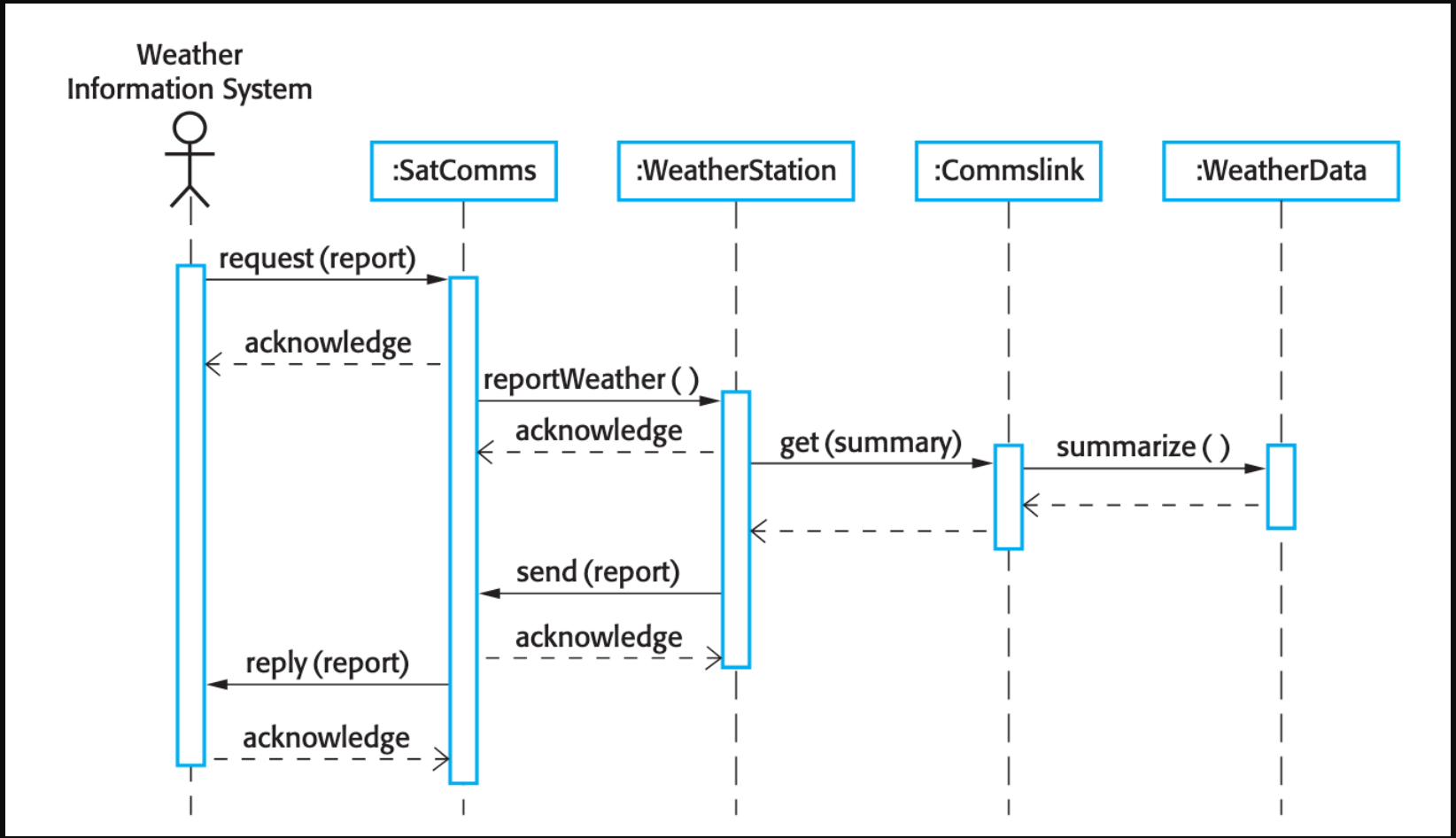

Architecture Design
Conceptual View
Logical View
High-level view
Main components
Satify requirements



Class diagrams

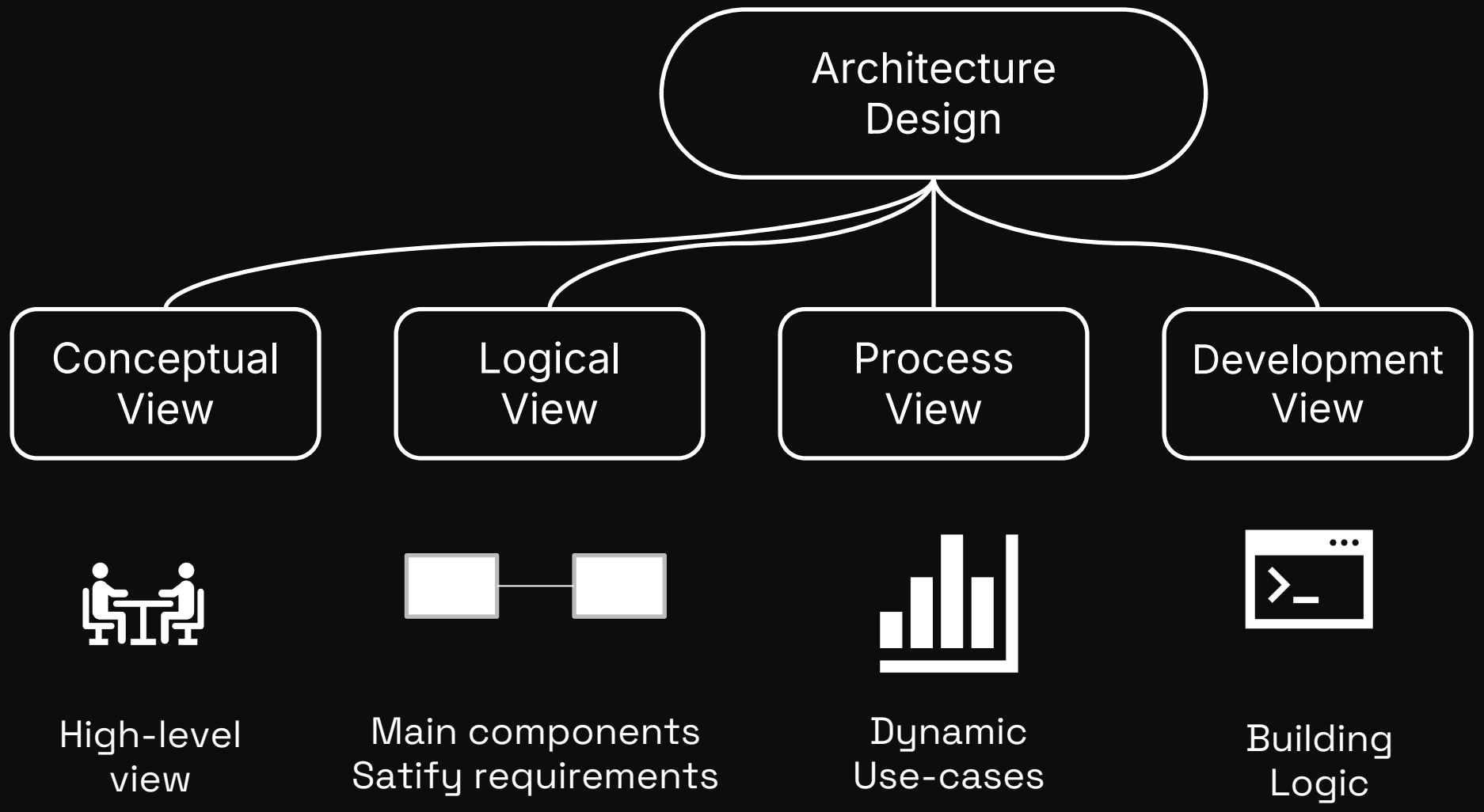
Most important components and their relationships

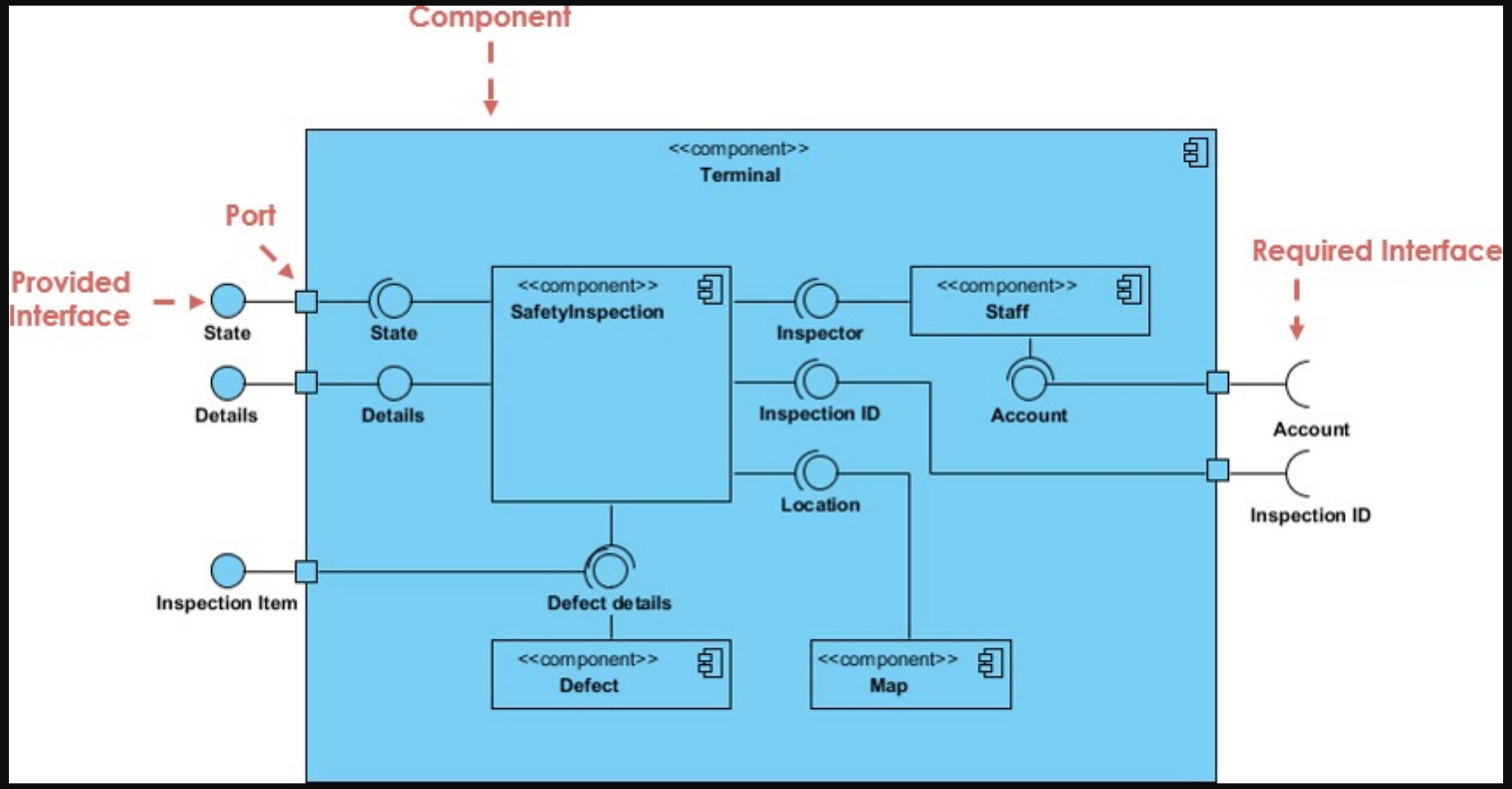

Architecture Design
Conceptual View
Logical View
Process View
High-level view
Main components
Satify requirements
Dynamic
Use-cases



Sequence diagrams

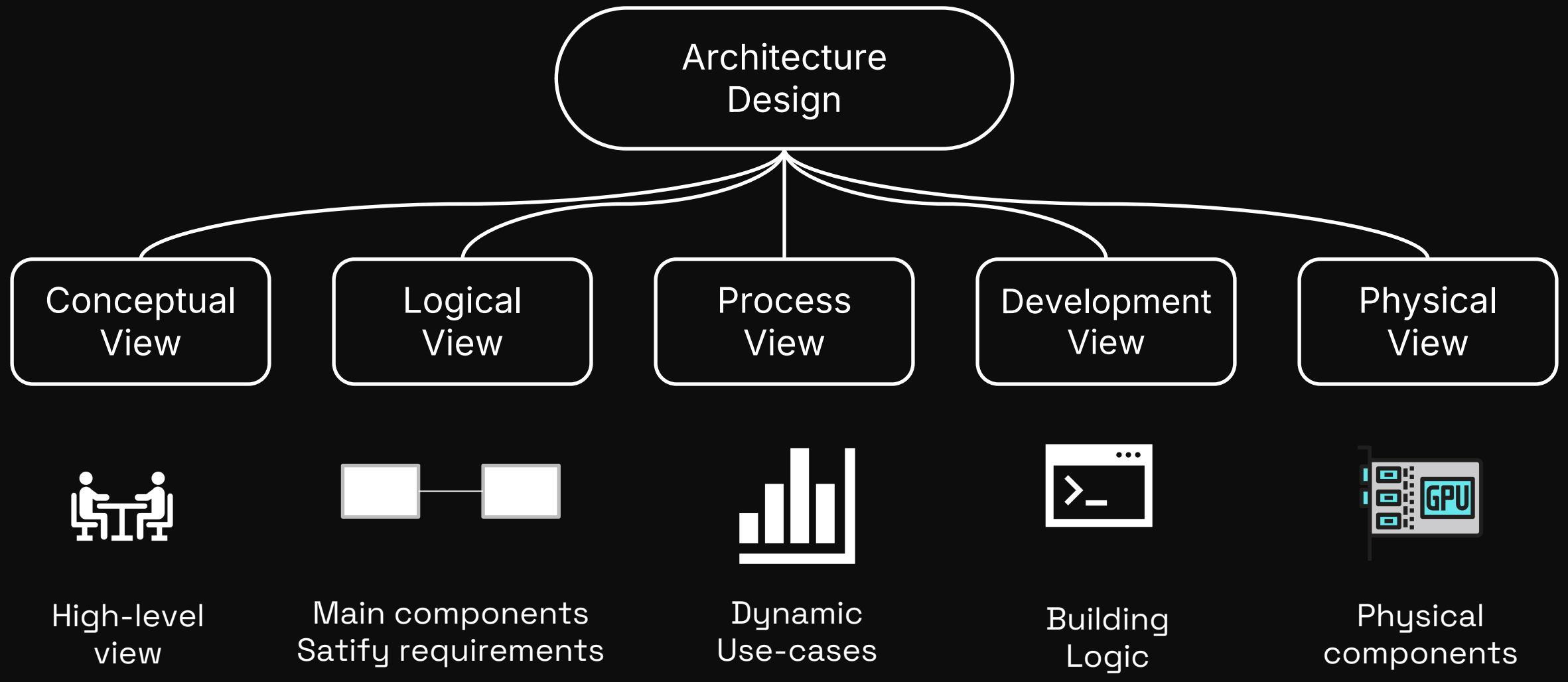
How components interact

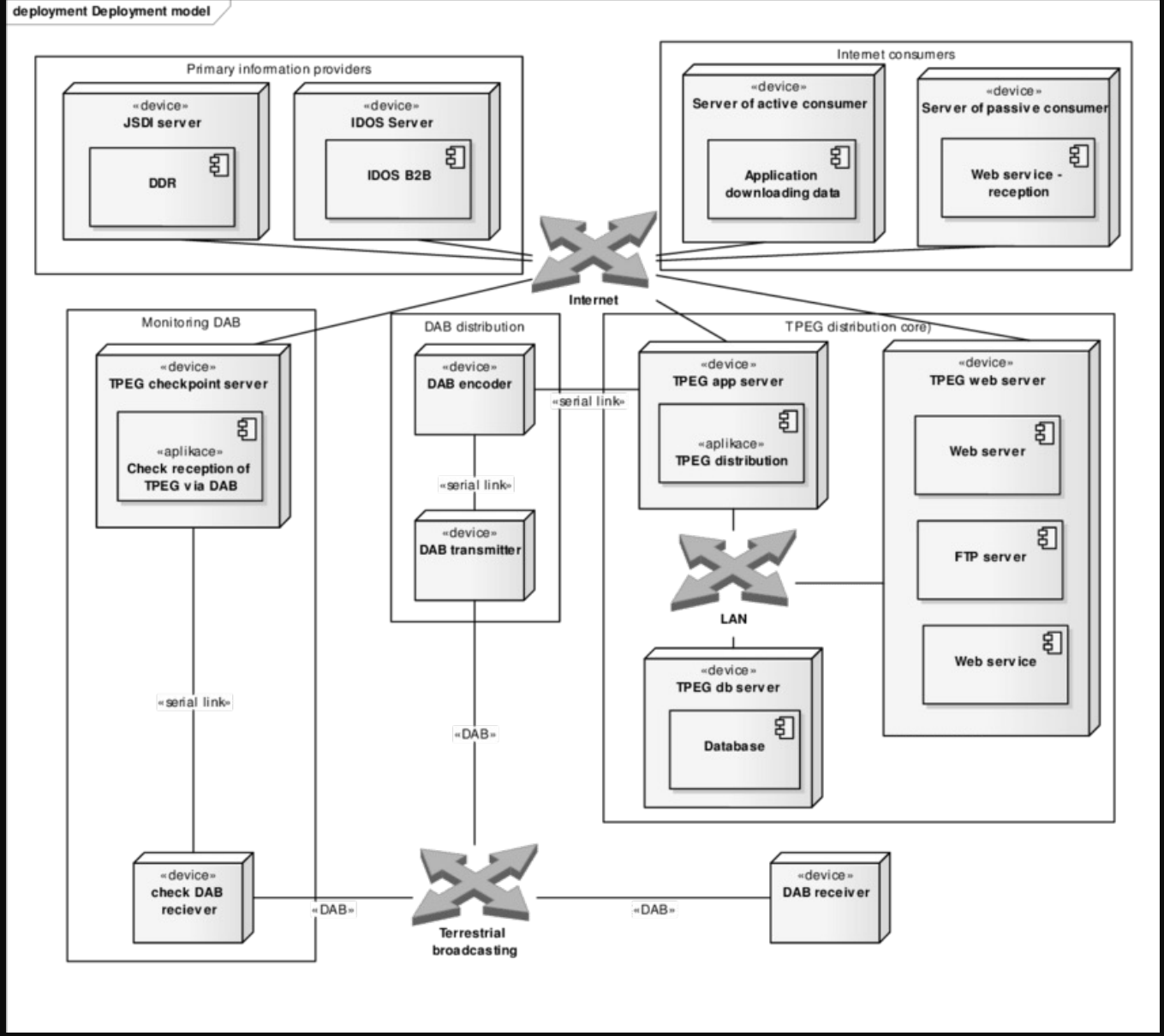

Architecture Design
Conceptual View
Logical View
Process View
Development View
High-level view
Main components Satify requirements
Dynamic Use-cases
Building Logic



Components diagrams

How to implement the system


Architecture Design
Conceptual View
Logical View
Process View
Development View
Physical View
High-level view
Main components Satify requirements
Dynamic Use-cases
Building Logic
GPU
Physical components



Deployment diagrams

Physical elements and communications

Architecture Design

Conceptual View

What is the system?

Business
End users

Logical View

What are the main components?

Developers

Process View

How do they Interact?

Developers
Performance devs

Development View

How is it built?

Developers

Physical View

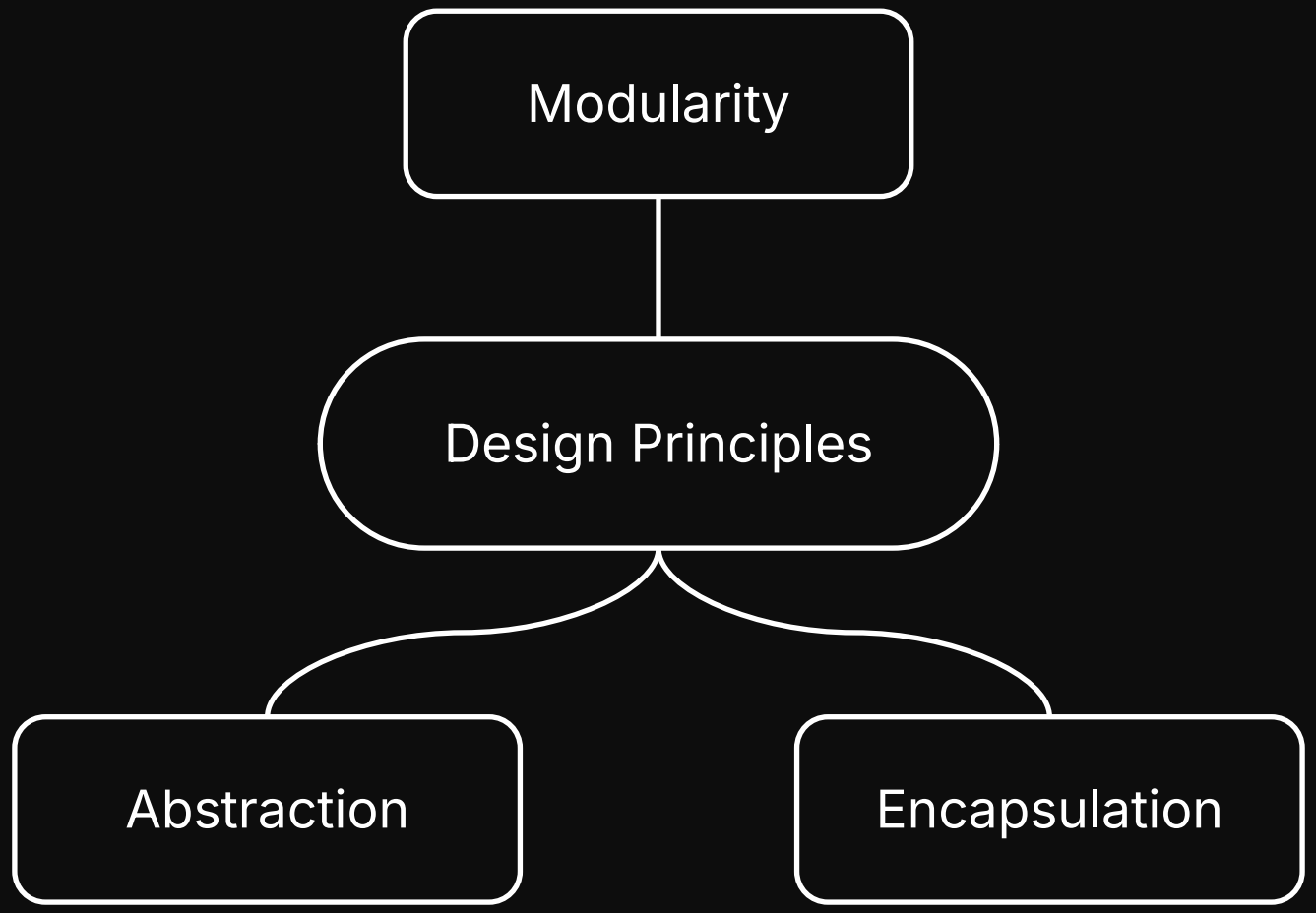
How is it deployed?

System architects

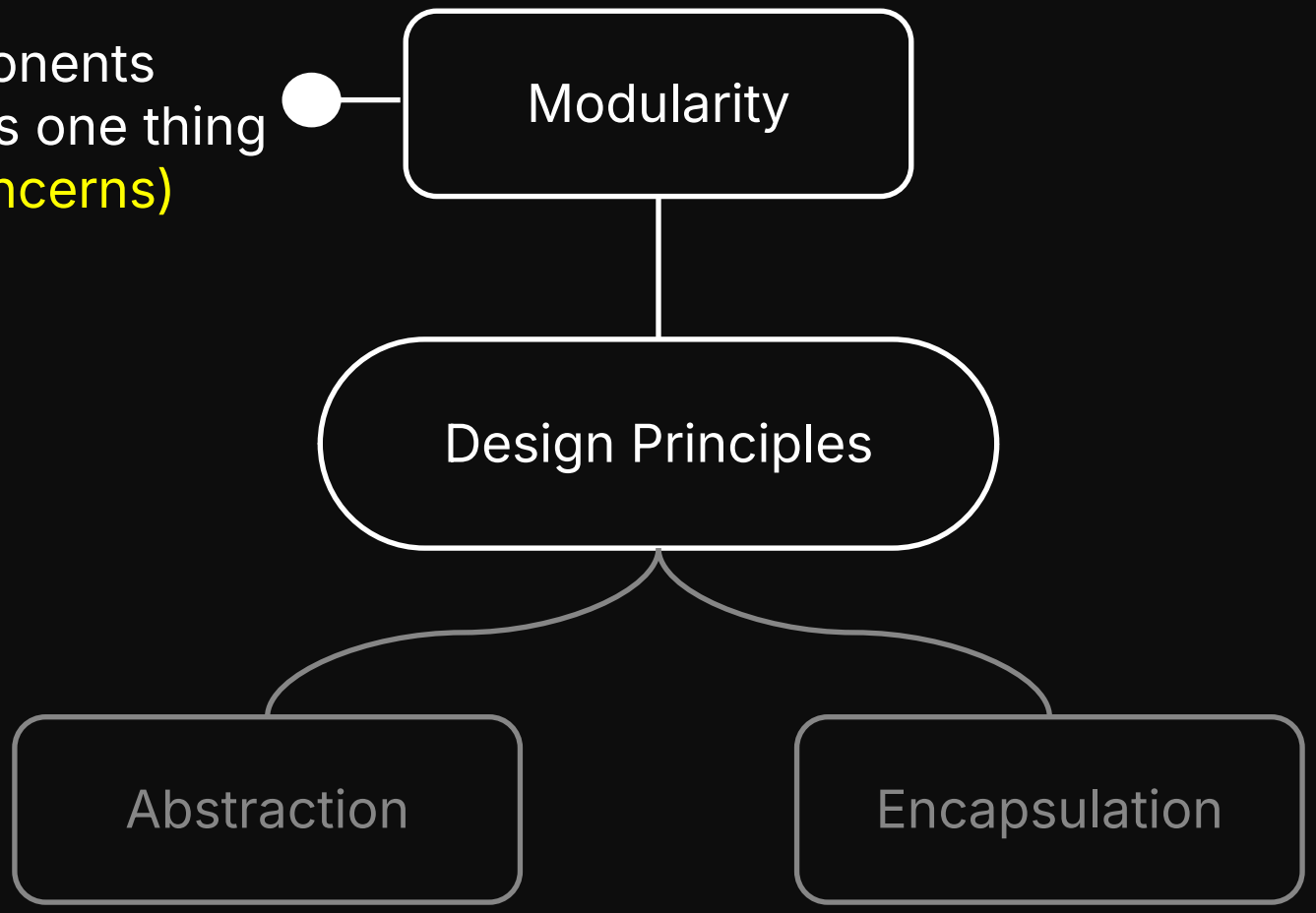
Main points

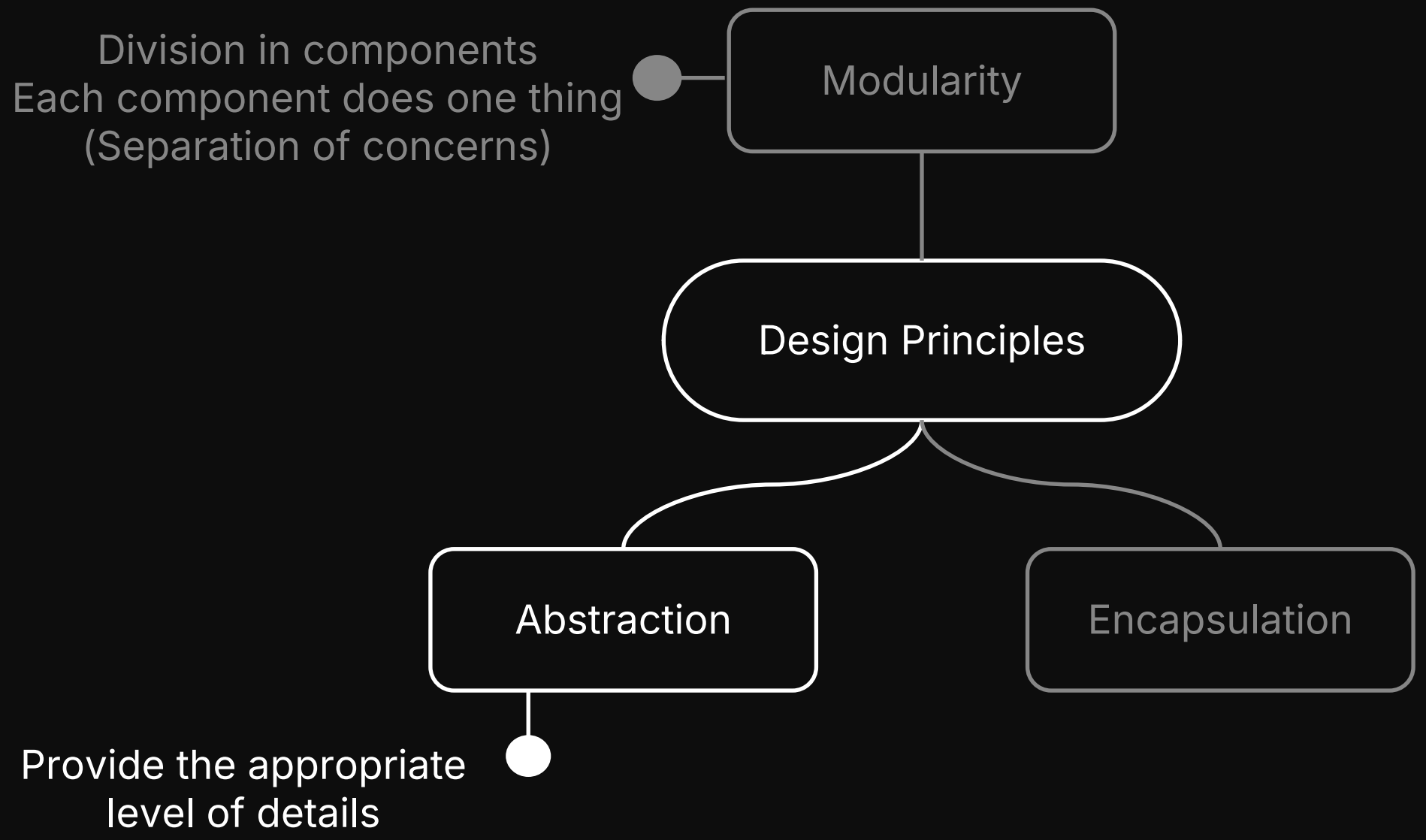
1 How to express architectural design? ⟶ Design Views

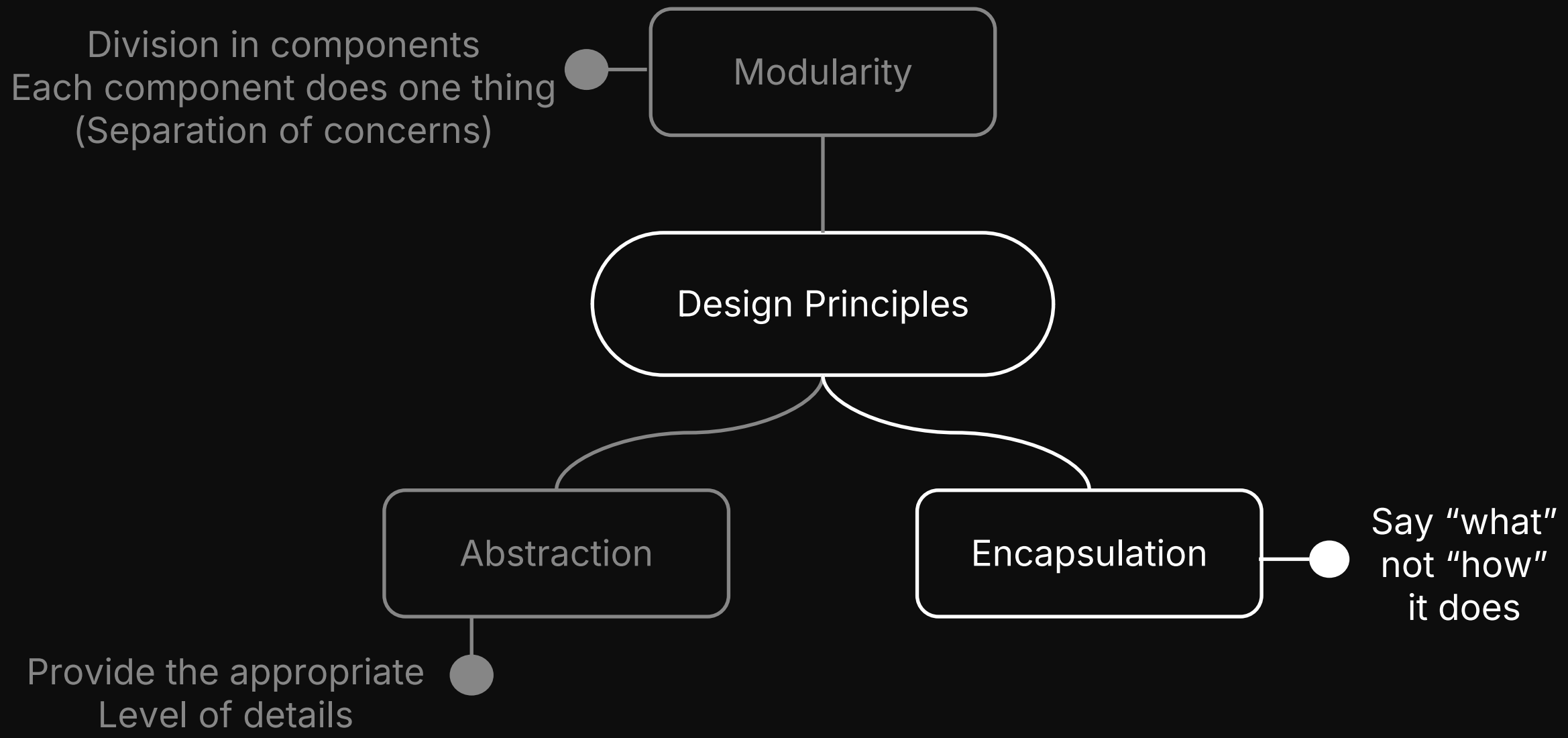
Design Principles


Modularity
Design Principles
Abstraction
Encapsulation

Division in components
Each component does one thing
(Separation of concerns)



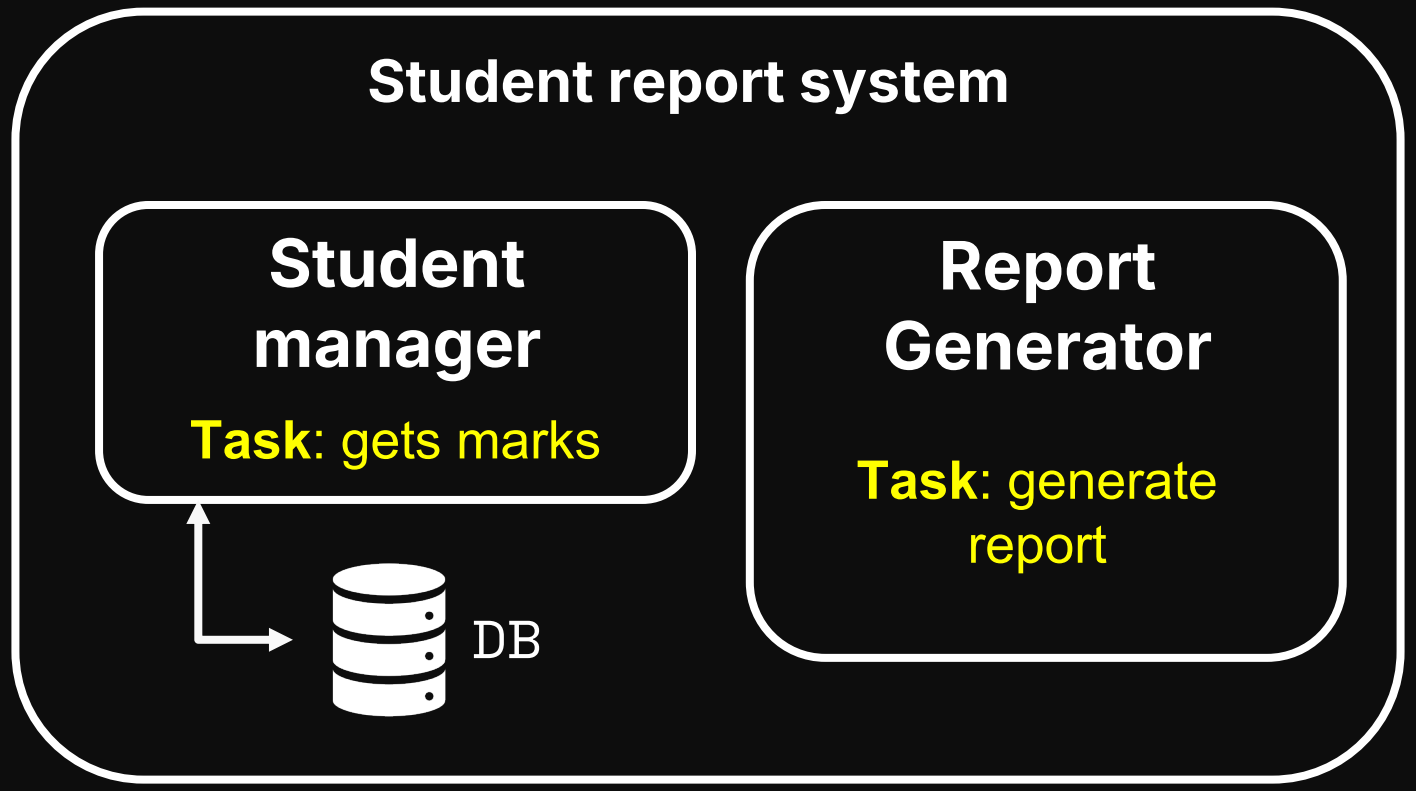

Division in components
Each component does one thing
(Separation of concerns)
Modularity
Design Principles
Abstraction
Encapsulation
Provide the appropriate
level of details


Division in components
Each component does one thing
(Separation of concerns)
Modularity
Design Principles
Abstraction
Encapsulation
Say “what”
not “how”
it does
Provide the appropriate
Level of details

Student report system

Task: Gets the average mark of a student
and generates a report

Modularity


Student report system
Student manager
Task: gets marks
Report Generator
Task: generate report
DB

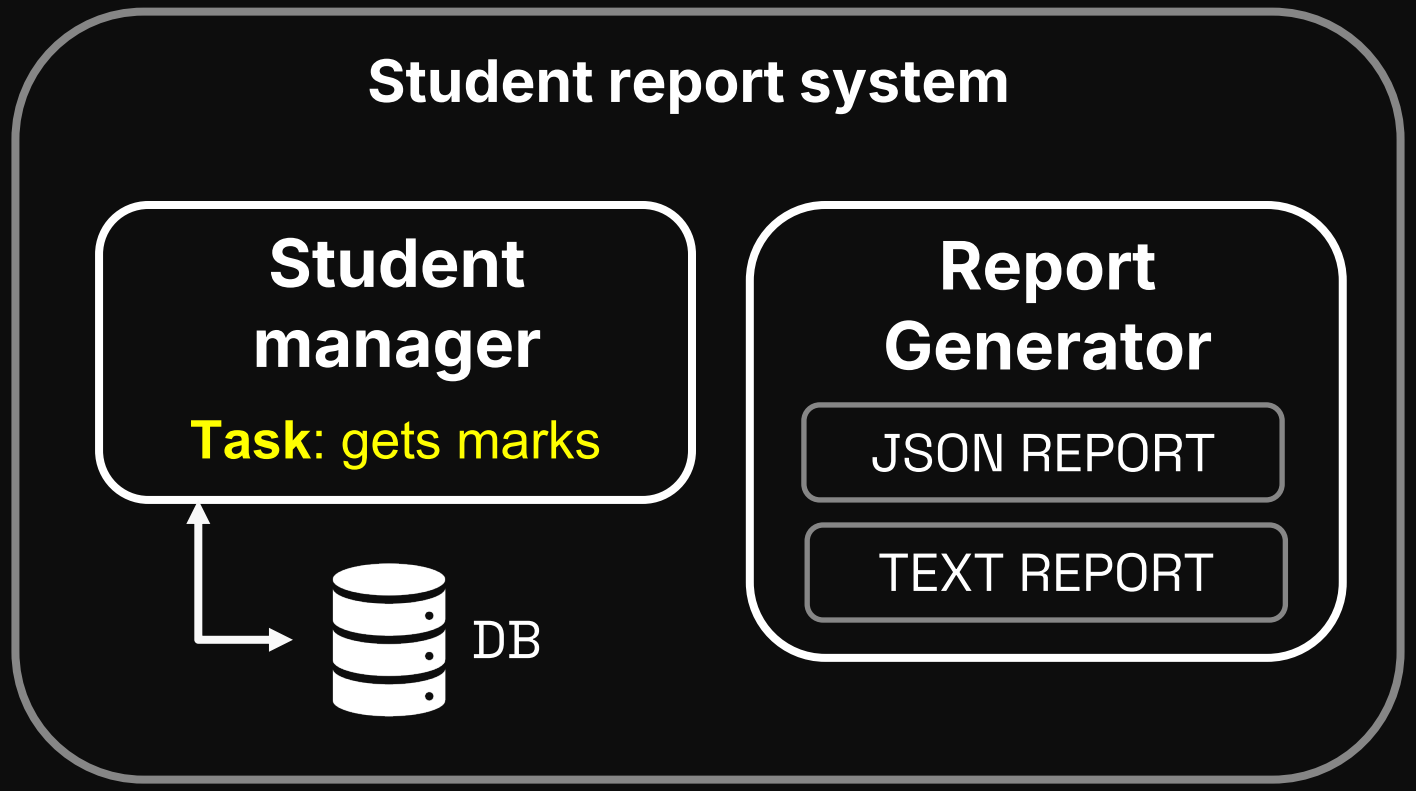
student_manager.py

```python
class StudentManager:
    def get_marks(self, student_id):
        db = Database.connect()
        return db.query("SELECT marks FROM students WHERE id = ?", student_id)
```

report_manager.py

```python
class ReportManager:
    def generate(self, student_id, marks):
        avg = sum(marks) / len(marks)
        status = "Pass" if avg >= 40 else "Fail"

        print("Student Report")
        print("ID:", student_id)
        print("Marks:", marks)
        print("Average:", avg)
        print("Result:", status)
```

Abstraction



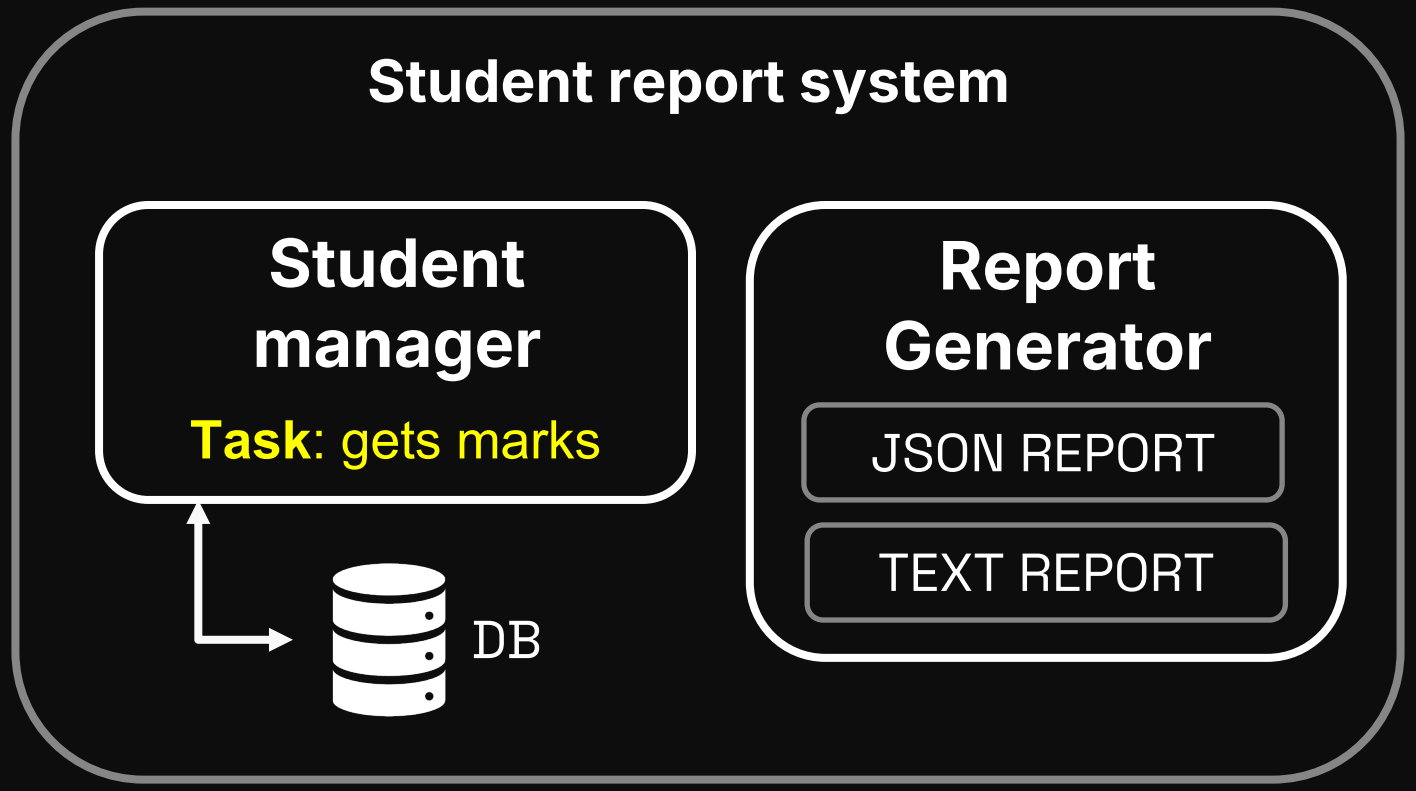
report_manager.py

```python
class JsonReportManager(ReportManager):
    def generate(self, student_id, marks):
        avg = sum(marks) / len(marks)

        return {
            "studentId": student_id,
            "marks": marks,
            "average": avg,
            "status": "Pass" if avg >= 40 else "Fail"
        }
```

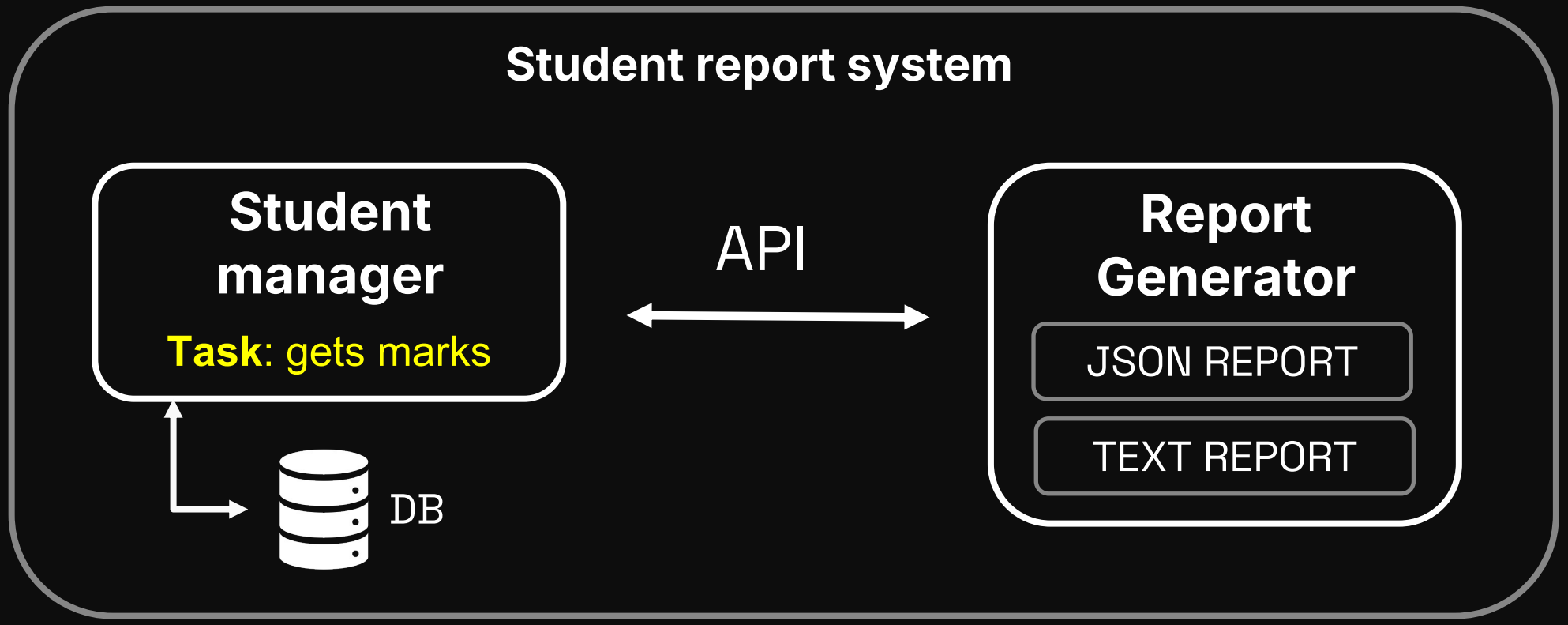
report_manager.py

```python
class TextReportManager(ReportManager):
    def generate(self, student_id, marks):
        avg = sum(marks) / len(marks)
        status = "Pass" if avg >= 40 else "Fail"

        report_text = (
            "Student Report\n"
            f"ID: {student_id}\n"
            f"Marks: {marks}\n"
            f"Average: {avg}\n"
            f"Result: {status}\n"
        )

        file = open(f"{student_id}_report.txt", "w")
        file.write(report_text)
        file.close()
```

Abstraction



Encapsulation



OTHER GUIDELINES

YAGNI

YAGNI

You Aren't Going to Need It

Only design features that are important for the current release of the software. Do not lose time doing stuff that may be (or not) useful in the future.

KISS®

KISS

Keep It Simple, Stupid

Things may be designed in the simplest possible way. Users are lazy, you don't want to complexify the interface

DRY

Don’t Repeat Yourself

Designed components should be reusable without significant efforts

Design guidelines

YAGNI

You aren't going to need it

Do the NEEDED

KISS

Keep it Simple, Stupid

Do it SIMPLE

DRY

Don't Repeat Yourself

Do it ONCE

Main points

1 How to express architectural design? ⟶ Design Views

2 Principles of good design

Non-functional requirements

Design decisions

What to prioritize?

Performance →

Critical components on the same device
Few large components vs many small
Replication and parallel execution

What to prioritize?

Performance

Critical components on the same device
Few large components vs many small
Replication and parallel execution

Security

Nested architecture, protect inner layers
High level of security in those layers

What to prioritize?

Performance → Critical components on the same device
Few large components vs many small
Replication and parallel execution

Security → Nested architecture, protect inner layers
High level of security in those layers

Safety → All safety-related elements in one component

Performance →
Critical components on the same device
Few large components vs many small
Replication and parallel execution

Security →
Nested architecture, protect inner layers
High level of security in those layers

Safety →
All safety-related elements in one component

Availability →
Include redundant components!

Performance →
Critical components on the same device
Few large components vs many small
Replication and parallel execution

Security →
Nested architecture, protect inner layers
High level of security in those layers

Safety →
All safety-related elements in one component

Availability →
Include redundant components!

Maintainability →
Many fine-grained, self-contained components
Easy to change

Main points

1 How to express architectural design? ⟶ Design Views

2 Principles of good design

3 Integration of non-functional requirements

SOLID

Principles for OOP design

S
O
L
I
D

Single Responsibility
A class should have only one job
(Example: data presentation and logic)

report_manager.py

```python
class JsonReportManager(ReportManager):
    def generate(self, student_id, marks):
        avg = sum(marks) / len(marks)

        return {
            "studentId": student_id,
            "marks": marks,
            "average": avg,
            "status": "Pass" if avg >= 40 else "Fail"
        }
```

Principles for OOP design

S
O
L
I
D

Open-Closed principle
Classes should be open for extensions and closed for modifications

payment.py

```python
class PaymentProcessor:
    def pay(self, payment_type: str, amount: float):
        if payment_type == "credit_card":
            print(f"Paying {amount} using Credit Card")
        elif payment_type == "paypal":
            print(f"Paying {amount} using PayPal")
        # adding crypto requires modifying this class ❌
        elif payment_type == "crypto":
            print(f"Paying {amount} using Crypto")
```

payment.py

```python
from abc import ABC, abstractmethod

class PaymentMethod(ABC):

    @abstractmethod
    def pay(self, amount: float) -> None:
        pass
```

Principles for OOP design

S
O
L
I
D

Liskov Substitution
Every subclass should be able to replace the parent

payment.py

```python
class PaymentMethod:
    def pay(self, amount: float) -> None:
        raise NotImplementedError
```

payment.py

```python
class CreditCardPayment(PaymentMethod):
    def pay(self, amount: float) -> None:
        print(f"Paying {amount} using Credit Card")


class CryptoPayment(PaymentMethod):
    def pay(self, amount: float) -> None:
        if amount > 1000:
            raise ValueError("Crypto payments limited to 1000")
        print(f"Paying {amount} using Crypto")
```

payment.py

```python
from abc import ABC, abstractmethod

class PaymentMethod(ABC):

    @abstractmethod
    def can_pay(self, amount: float) -> bool:
        pass

    @abstractmethod
    def pay(self, amount: float) -> None:
        pass
```

payment.py

```python
class CreditCardPayment(PaymentMethod):
    def can_pay(self, amount: float) -> bool:
        return True

    def pay(self, amount: float) -> None:
        print(f"Paying {amount} using Credit Card")


class CryptoPayment(PaymentMethod):
    def can_pay(self, amount: float) -> bool:
        return amount <= 1000

    def pay(self, amount: float) -> None:
        print(f"Paying {amount} using Crypto")
```

Principles for OOP design

S
O
L
I
D

Interface segregation principle
Many small interfaces are better than a big one

payment.py

```python
from abc import ABC, abstractmethod

class PaymentMethod(ABC):

    @abstractmethod
    def pay(self, amount: float) -> None:
        pass

    @abstractmethod
    def refund(self, amount: float) -> None:
        pass

    @abstractmethod
    def generate_invoice(self) -> None:
        pass
```

Bad solution!

payment.py

```python
class CryptoPayment(PaymentMethod):
    def pay(self, amount: float) -> None:
        print(f"Paying {amount} with Crypto")

    def refund(self, amount: float) -> None:
        raise NotImplementedError("Crypto refunds not supported")  # 🚨

    def generate_invoice(self) -> None:
        print("Generating crypto invoice")
```

payment.py

```python
class Payable(ABC):
    @abstractmethod
    def pay(self, amount: float) -> None:
        pass


class Refundable(ABC):
    @abstractmethod
    def refund(self, amount: float) -> None:
        pass


class Invoicable(ABC):
    @abstractmethod
    def generate_invoice(self) -> None:
        pass
```

payment.py

```python
class CreditCardPayment(Payable, Refundable, Invoicable):
    def pay(self, amount: float) -> None:
        print(f"Paying {amount} with Credit Card")

    def refund(self, amount: float) -> None:
        print(f"Refunding {amount} to Credit Card")

    def generate_invoice(self) -> None:
        print("Generating credit card invoice")


class CryptoPayment(Payable, Invoicable):
    def pay(self, amount: float) -> None:
        print(f"Paying {amount} with Crypto")

    def generate_invoice(self) -> None:
        print("Generating crypto invoice")
```

Principles for OOP design

S
O
L
I
D

Dependency inversion principle
Entities should depend on abstractions

payment.py

```python
class StripePayment:
    def pay(self, amount: float) -> None:
        print(f"Paying {amount} via Stripe")


class CheckoutService:
    def __init__(self):
        self.payment = StripePayment()  # ❌ hard dependency

    def checkout(self, amount: float) -> None:
        self.payment.pay(amount)
```

payment.py

```python
from abc import ABC, abstractmethod

class PaymentGateway(ABC):

    @abstractmethod
    def pay(self, amount: float) -> None:
        pass

class CheckoutService:
    def __init__(self, payment_gateway: PaymentGateway):
        self.payment_gateway = payment_gateway

    def checkout(self, amount: float) -> None:
        self.payment_gateway.pay(amount)
```

Main points

1 How to express architectural design? ⟶ Design Views

2 Principles of good design

3 Integration of non-functional requirements

4 Principles of OOP design guidelines

Thanks!

References:
Sommerville Chapter 6